



Vietnam National University of HCMC
International University
School of Computer Science and Engineering



Object Oriented Programming

Classes & Objects

Instructor: Le Thi Ngoc Hanh, Ph.D

ltnhanh@hcmiu.edu.vn

Semester – Spring 2026

Lecture Objectives

- ☾ What a class really represents.
- ☾ What an object really is.
- ☾ How objects live in memory.
- ☾ How objects are created.
- ☾ How to model a simple real-world entity.

Procedural vs Object Thinking

Traditional Procedural Model

- ☾ Program = data + functions
- ☾ Data is separate from logic
- ☾ Functions manipulate global data

Object-Oriented Model

- ☾ Program = interacting objects
- ☾ Each object:
 - Has its own state (data)
 - Controls its own behavior

Definition of Class

- ☾ A class is a user-defined reference type that specifies the structure and characteristics shared by its objects.
- ☾ A class defines:
 - Instance variables (state)
 - Constructors (initialization)
 - Methods (behavior)
- ☾ A class is a logical definition and does not represent a runtime entity.

Definition of Class

Class Declaration syntax:

```
[Access Modifier] class ClassName {  
    // fields  
    // methods  
}
```

```
public class Student {  
    String name;  
    int age;  
}
```

Class vs Objects



Class



Objects

```
public class Student {  
    String name;  
    int age;  
}
```



Real student?

Definition of Object

★ An object (instance) is:

- An instance of a class
- A runtime instance of a class

★ An object has:

- Identity (unique memory address)
- State (values of instance variables)

Creating Objects

★ General syntax:

```
ClassName variableName = new ClassName();
```

```
Student s = new Student();
```

★ The statement performs **three operations**:

- Allocates memory in the heap
- Calls the constructor
- Assigns the object reference to a variable

```
Student s1 = new Student("Tommy", 20);
```

```
Student s2 = new Student("Tommy", 20);
```

Object Identity and State

- ✧ Identity refers to the object's unique memory address.
- ✧ State refers to the values stored in instance variables.
- ✧ Two objects may have identical state but different identity.
- ✧ Object equality based on identity differs from equality based on data.
- ✧ Each object maintains independent state.

```
Student s1 = new Student("Tommy", 20);  
Student s2 = new Student("Tommy", 20);
```

Structure of a Well-Defined Class

- ☾ Instance variables represent intrinsic properties.
- ☾ Fields are typically declared private to protect state.
- ☾ Constructors initialize object state.
- ☾ The keyword `this` refers to the current object instance.
- ☾ Proper initialization ensures object consistency.

☾ Sample code:

```
public class Student {  
    private String name;  
    private int age;  
  
    public Student(String name, int age) {  
        this.name = name;  
        this.age = age;  
    }  
}
```

Multiple Object Instantiation

- ☾ One class can create many objects.
- ☾ Each object has independent memory allocation.
- ☾ Modifying one object does not affect another.
- ☾ Object state is stored separately per instance.

```
Student s1 = new Student("Tommy", 20);  
Student s2 = new Student("Lucia", 20);
```

Memory model and Object Lifecycle

- ☾ Stack memory stores:
 - Local variables
 - Object references
- ☾ Heap memory stores:
 - Objects
 - Instance variables
- ☾ Objects remain in heap until no references point to them.
- ☾ Multiple reference variables may point to the same object.
- ☾ Stack variables disappear after method execution

Example:

```
Student s = new Student("John", 20);
```

Stack vs Heap

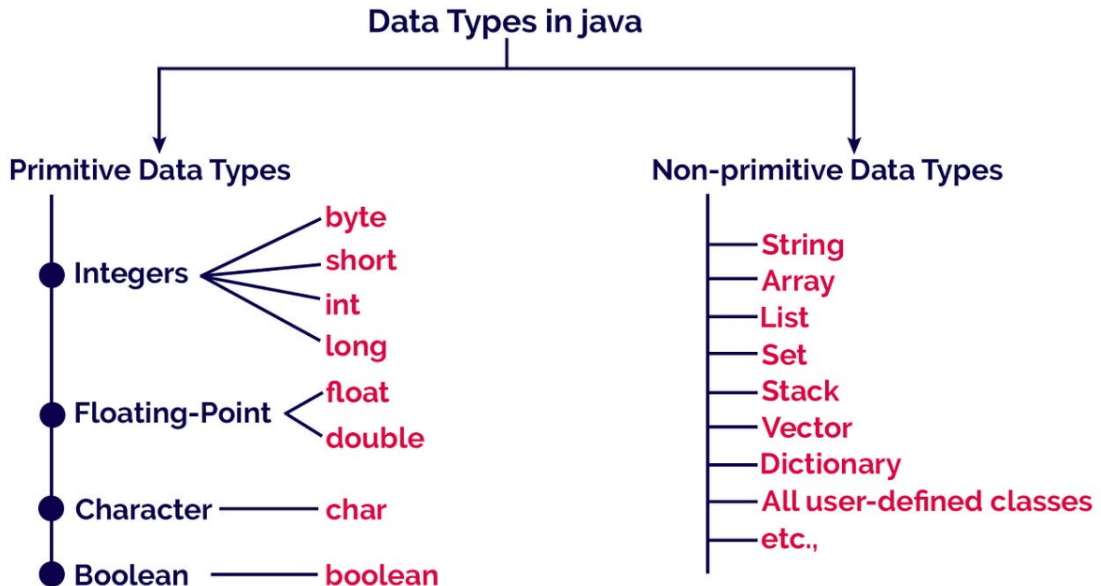
STACK

Local variables
Method parameters
Object references

HEAP

Objects
Instance variables

Recall - Java Data Types



A Class Defines a New Reference Type!

Instance Variables vs Local Variables

★ Instance variables:

- Represent object state
- Shared by all methods of the class
- Have default values

★ Local variables:

- Temporary
- Used for computation
- No default values

Instance Variables vs Local Variables

Example:

```
public class Student {  
    private String name;    // instance variable  
  
    public Student(String name) {  
        String temp = name;    // local variable  
        this.name = temp;  
    }  
}
```

Instance Variables vs Local Variables

Where Are Instance Variables Declared?

- Inside the class
- Outside all methods

```
public class ClassName {  
    // instance variables here  
}
```

```
public class Student {  
    String name;  
    int age;  
    double gpa;  
}
```

Instance Variables vs Local Variables

- ★ When an object is created, instance variables are automatically initialized.

Type	Default value
int	0
Double	0.0
Boolean	False
Reference types	null

- ★ Every object has its own separate copy of instance variables.

Accessing Instance Variables

☾ Syntax:

```
objectName.variableName
```

```
Student s1 = new Student();  
s1.name = "Alice";
```

Constructors and Object Initialization

- ☾ When an object of a class is created, its instance variables are initialized by default.
- ☾ Each class can provide a constructor that initializes an object of a class when the object is created.
- ☾ Java requires a constructor call for every object that is created.
- ☾ A constructor *must* have the same name as the class.
- ☾ Constructor ensures the object begins in a consistent and meaningful state.

Constructors and Object Initialization

Recall example:

```
public class Student {  
    String name;  
    int age;  
}  
Student s = new Student();
```

Constructors and Object Initialization

Example:

```
public class Student {
```

```
    private String name;
```

```
    private int age;
```

```
    public Student(String name, int age) {
```

```
        this.name = name;
```

```
        this.age = age;
```

```
    }
```

```
}
```



```
public class Main {
```

```
    public static void main(String[] args) {
```

```
        Student s1 = new Student("Tommy", 20);
```

```
    }
```

```
}
```